



Universidade Federal
do Rio de Janeiro

Escola Politécnica

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

Projeto de Graduação apresentado ao
Curso de Engenharia Mecânica da Escola
Politécnica, Universidade Federal do Rio
de Janeiro, como parte dos requisitos ne-
cessários à obtenção do título de Engenheiro
Mecânico.

Orientador: Gustavo Rabello dos Anjos

Rio de Janeiro
Fevereiro de 2026

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO
DE ENGENHARIA MECÂNICA DA ESCOLA POLITÉCNICA DA UNIVERSI-
DADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NE-
CESSÁRIOS PARA A OBTENÇÃO DO GRAU DE ENGENHEIRO MECÂNICO

Autor:

Luiz Gustavo Santos Farias

Orientador:

Prof. Gustavo Rabello dos Anjos, D. Sc.

Examinador:

Prof. Examinador 1, D. Sc.

Examinador:

Prof. Examinador 2, D. Sc.

Rio de Janeiro
Fevereiro de 2026

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Escola Politécnica - Departamento de Engenharia Mecânica

Centro de Tecnologia, bloco G, sala G-204, Cidade Universitária

Rio de Janeiro - RJ CEP 21941-909

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmear ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

DEDICATÓRIA

Dedico este trabalho aos meus pais, amigos e mestres que me apoiaram nesta jornada.

AGRADECIMENTO

Agradeço primeiramente à minha família, pelo suporte incondicional. Ao meu orientador, Prof. Gustavo Rabello dos Anjos, pela guiaça e confiança. Aos colegas da UFRJ, pelo companheirismo. E a todos que contribuíram direta ou indiretamente para a realização deste projeto.

RESUMO

A simulação numérica constitui ferramenta estratégica para análise e projeto em Engenharia Mecânica, porém seu uso em ambiente educacional ainda é limitado por custos de licenciamento, requisitos de infraestrutura e complexidade de configuração. Este trabalho apresenta o *MinervaMesh*, uma plataforma de simulação numérica executada integralmente em navegador, concebida para reduzir barreiras de acesso sem comprometer o rigor técnico. A solução adota arquitetura *client-side* baseada em *WebAssembly* e *PyScript*, permitindo a execução de rotinas em Python científico para solução de Equações Diferenciais Parciais por Método dos Elementos Finitos (MEF) e Método das Diferenças Finitas (MDF). A metodologia de validação prioriza casos com solução analítica conhecida para verificação de precisão, estabilidade e convergência, complementados por casos adicionais representativos de fluidodinâmica e transferência de calor. Os resultados indicam que a plataforma preserva consistência numérica e apresenta potencial de aplicação em ensino, prototipagem e demonstrações acadêmicas de mecânica computacional.

Palavras-Chave: Método dos Elementos Finitos, Método das Diferenças Finitas, Simulação Web, Mecânica Computacional, Ensino de Engenharia.

ABSTRACT

Numerical simulation is a core tool for analysis and design in Mechanical Engineering; however, its broader adoption in education is still constrained by software licensing costs, infrastructure requirements, and setup complexity. This work presents *MinervaMesh*, a browser-based numerical simulation platform designed to reduce access barriers while maintaining technical rigor. The proposed system follows a fully *client-side* architecture built on *WebAssembly* and *PyScript*, enabling scientific Python workflows for solving Partial Differential Equations with the Finite Element Method (FEM) and the Finite Difference Method (FDM). The validation strategy prioritizes benchmark cases with known analytical solutions to assess accuracy, stability, and convergence, followed by additional representative fluid-flow and heat-transfer cases. Results indicate that the platform preserves numerical consistency and is suitable for educational use, rapid prototyping, and academic demonstrations in computational mechanics.

Key-words: Finite Element Method, Finite Difference Method, Web Simulation, Computational Mechanics, Engineering Education.

SIGLAS

UFRJ - Universidade Federal do Rio de Janeiro

MEF - Método dos Elementos Finitos

MDF - Método das Diferenças Finitas

CFD - *Computational Fluid Dynamics* (Dinâmica dos Fluidos Computacional)

WASM - WebAssembly

SUPG - *Streamline Upwind/Petrov-Galerkin*

Sumário

1	Introdução	1
1.1	Introdução	1
1.2	Objetivos	2
1.3	Justificativa	3
1.4	Organização do Trabalho	3
2	Revisão Bibliográfica	4
2.1	Introdução ao Capítulo	4
2.2	Problemas Térmicos e Equações Governantes	4
2.2.1	Condução de Calor	4
2.2.2	Convecção e Acoplamento com Escoamento	4
2.2.3	Radiação Térmica	5
2.3	Métodos Numéricos para EDPs	5
2.3.1	Método das Diferenças Finitas (MDF)	5
2.3.2	Método dos Elementos Finitos (MEF)	5
2.4	Bases Históricas e Consolidação do MEF/CFD	6
2.5	Trabalhos Correlatos (Ênfase em TCCs Semelhantes)	6
2.6	Síntese da Revisão e Lacuna Atacada	7
3	Fundamentação Teórica	8
3.1	Introdução ao Capítulo	8
3.2	Equações Governantes	8
3.2.1	Condução e Convecção Térmica	9
3.2.2	Escoamento de Fluidos (Navier-Stokes)	9
3.3	Transição Discreta: O Método das Diferenças Finitas (MDF)	10

3.4	O Método dos Elementos Finitos (MEF)	10
3.4.1	Da Forma Forte à Forma Fraca	11
3.4.2	O Método de Galerkin	12
3.4.3	Elementos Triangulares Lineares (P1)	13
3.4.4	Matrizes Elementares	14
3.4.5	Montagem Global	15
3.5	Formulação Vorticidade-Corrente	15
3.5.1	Definições	15
3.5.2	Equação de Transporte da Vorticidade	16
3.5.3	Sistema Acoplado e Solução Iterativa	16
3.5.4	Matriz de Convecção e Estabilização SUPG	16
3.6	Integração Temporal	17
3.6.1	Método θ Generalizado	17
3.6.2	Esquema Semi-Implicito Adotado	18
3.6.3	Condição de Contorno de Vorticidade: Fórmula de Thom	18
3.7	Síntese do Capítulo	19
4	Desenvolvimento e Arquitetura	20
4.1	Visão Geral da Plataforma	20
4.2	Justificativa Tecnológica	21
4.2.1	PyScript e WebAssembly	21
4.2.2	Computação <i>Client-Side</i>	22
4.2.3	Comparação com Alternativas	23
4.3	Arquitetura de Software	23
4.3.1	Organização de Diretórios	23
4.3.2	Fluxo de Execução de uma Simulação	24
4.4	Módulos de Simulação MEF	26
4.4.1	Condução Permanente 1D	26
4.4.2	Condução Transiente 1D com $k(x)$ Variável	27
4.4.3	Convecção-Difusão 1D com Estabilização SUPG	28
4.4.4	Viga 1D — Análise de Flexão (Euler-Bernoulli)	29
4.4.5	Vibração 1D — Problema de Autovalor	30
4.4.6	Transferência de Calor 2D	31

4.4.7	Escoamento Potencial 2D	33
4.4.8	Navier-Stokes 2D (Vorticidade-Corrente)	33
4.5	Síntese do Capítulo	36
5	Resultados e Validação	38
5.1	Introdução ao Capítulo	38
6	Conclusões	39
6.1	Conclusões	39
6.2	Trabalhos Futuros	39
	Bibliografia	40
A	O que é um apêndice	45
B	Encadernação do Projeto de Graduação	46
C	O que é um anexo	48

Lista de Figuras

4.1	Tela inicial do <i>MinervaMesh</i> com o menu de simulações disponíveis. .	21
4.2	Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.	25
4.3	Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.	27
4.4	Evolução temporal do campo de temperatura com condutividade variável $k(x)$	28
4.5	Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.	29
4.6	Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.	30
4.7	Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.	31
4.8	Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).	32
4.9	Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.	33
4.10	Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.	35
4.11	Navier-Stokes 2D — linhas de corrente para escoamento ao redor de um cilindro.	36
B.1	Encadernação do projeto de graduação.	47

Lista de Tabelas

4.1	Comparação entre plataformas de simulação numérica para ensino. . .	23
-----	---	----

Capítulo 1

Introdução

1.1 Introdução

A simulação numérica consolidou-se como instrumento central da engenharia contemporânea, complementando abordagens teóricas e experimentais na investigação de fenômenos físicos complexos. No contexto da Engenharia Mecânica, a Dinâmica dos Fluidos Computacional (*Computational Fluid Dynamics* – CFD) e a modelagem de transferência de calor viabilizam a análise de problemas governados por Equações Diferenciais Parciais (EDPs), muitas vezes inviáveis por solução analítica direta ou por experimentação em escala de laboratório [1, 2]. Essa capacidade preditiva tem papel relevante no dimensionamento, na otimização e na verificação de desempenho de sistemas térmicos e fluidodinâmicos.

Apesar dos avanços metodológicos, o acesso a ambientes de simulação permanece limitado por barreiras econômicas e operacionais. Soluções comerciais amplamente utilizadas exigem licenciamento oneroso, infraestrutura computacional específica e configuração de ambiente com elevado custo de entrada, sobretudo em cenários de ensino e pesquisa inicial. Como consequência, a formação em métodos numéricos frequentemente fica condicionada à disponibilidade de laboratório e suporte técnico especializado, o que reduz a reprodutibilidade e a acessibilidade das atividades acadêmicas [3].

Neste contexto, este trabalho propõe o *MinervaMesh*, uma plataforma de simulação numérica executada integralmente no navegador, com arquitetura *client-*

side. A solução combina Python científico com tecnologias web modernas, notadamente *WebAssembly* e *PyScript*, permitindo a execução local de rotinas de MEF e MDF sem instalação de dependências pelo usuário [4, 5]. A contribuição principal não se restringe à implementação computacional, mas inclui a organização de um ambiente didático de engenharia para formulação, solução e análise de problemas térmicos e fluidodinâmicos.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver e validar o **MinervaMesh**, uma plataforma computacional baseada na web para solução de EDPs de interesse em Engenharia Mecânica, operando integralmente no lado do cliente (*client-side*).

Os objetivos específicos são:

1. Implementar métodos numéricos clássicos, com ênfase no Método dos Elementos Finitos (MEF) e uso complementar do Método das Diferenças Finitas (MDF), empregando bibliotecas científicas Python no ambiente web [5].
2. Desenvolver uma interface interativa para pré-processamento, solução e pós-processamento, incluindo parametrização de malha e visualização de campos de interesse.
3. Validar a plataforma, prioritariamente, por meio de casos com solução analítica conhecida e, em seguida, demonstrar sua aplicação em casos adicionais de engenharia, com foco em mecânica dos fluidos e transferência de calor.
4. Consolidar um fluxo de trabalho reprodutível para problemas de:
 - Escoamentos potenciais;
 - Problemas de advecção-difusão (com estabilização SUPG) [6];
 - Equações de Navier-Stokes para escoamentos laminares incompressíveis.

1.3 Justificativa

A relevância do trabalho reside na articulação entre rigor numérico e acessibilidade computacional. Ao viabilizar simulações diretamente no navegador, reduz-se o atrito inicial de aprendizado e amplia-se o potencial de uso em disciplinas de graduação, monitorias e estudos independentes. Adicionalmente, a explicitação da formulação matemática e de sua implementação computacional fortalece o caráter acadêmico da proposta, permitindo que a plataforma seja utilizada tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos [7, 8].

1.4 Organização do Trabalho

Este trabalho está estruturado da seguinte forma:

- O **Capítulo 2** (Revisão Bibliográfica) apresenta o histórico e o estado da arte dos temas que sustentam o trabalho, incluindo problemas térmicos, MDF, MEF e trabalhos correlatos.
- O **Capítulo 3** (Fundamentação Teórica) apresenta o embasamento matemático necessário para a discretização de EDPs via MEF.
- O **Capítulo 4** (Desenvolvimento e Arquitetura) detalha a implementação do *MinervaMesh*, com ênfase na arquitetura *client-side* e PyScript.
- O **Capítulo 5** (Resultados e Validação) apresenta a verificação numérica em casos com solução analítica e casos de aplicação.
- O **Capítulo 6** (Conclusão) resume as contribuições e propõe trabalhos futuros.

Capítulo 2

Revisão Bibliográfica

2.1 Introdução ao Capítulo

Este capítulo apresenta a revisão bibliográfica que fundamenta o desenvolvimento do *MinervaMesh*, com foco em problemas térmicos e de mecânica dos fluidos resolvidos por Métodos Numéricos, em especial o Método das Diferenças Finitas (MDF) e o Método dos Elementos Finitos (MEF). A revisão está organizada em quatro blocos: fundamentos de transferência de calor, bases dos métodos numéricos, marcos do MEF/CFD e trabalhos correlatos.

2.2 Problemas Térmicos e Equações Governantes

2.2.1 Condução de Calor

A condução é descrita pela Lei de Fourier e pela equação da difusão térmica. Em regime estacionário, a formulação conduz às equações de Laplace e Poisson; em regime transiente, inclui-se o termo temporal associado ao armazenamento de energia [9, 10, 11, 12]. Esses modelos são a base de problemas clássicos de engenharia, como dissipação em componentes eletrônicos e análise térmica de discos de freio.

2.2.2 Convecção e Acoplamento com Escoamento

Quando há interação sólido-fluido, a convecção influencia a taxa de remoção de calor e exige o acoplamento entre equações de energia e de escoamento. A formulação

por volumes/elementos finitos para problemas convectivo-difusivos é amplamente discutida na literatura de CFD [1, 2, 13, 14, 15].

2.2.3 Radiação Térmica

Em aplicações de alta temperatura, os termos radiativos podem se tornar relevantes no balanço térmico global [9, 11]. Embora o escopo principal deste trabalho esteja em condução e convecção, a revisão considera radiação como extensão natural para trabalhos futuros.

2.3 Métodos Numéricos para EDPs

2.3.1 Método das Diferenças Finitas (MDF)

O MDF aproxima derivadas por expansões de Taylor, convertendo EDPs em sistemas algébricos. Sua implementação é direta em malhas estruturadas e, por isso, ele é frequentemente adotado em validações iniciais e em discretização temporal de problemas transientes [2, 16].

Esquemas explícitos e implícitos (Euler e Crank-Nicolson) apresentam diferentes compromissos entre custo computacional, estabilidade e precisão. Em problemas dominados por difusão, esquemas implícitos são preferidos por robustez numérica [2, 1].

2.3.2 Método dos Elementos Finitos (MEF)

O MEF oferece maior flexibilidade geométrica por meio de malhas não estruturadas e formulação variacional. Essa característica é decisiva em domínios complexos, comuns em aplicações reais de engenharia térmica e de fluidos [7, 17, 8, 18, 19, 20, 21].

A formulação de Galerkin transforma a EDP na forma fraca e permite a montagem de matrizes globais de rigidez, massa e convecção. Para elementos lineares triangulares, a implementação é eficiente e adequada a plataformas computacionais didáticas [8, 18, 7].

2.4 Bases Históricas e Consolidação do MEF/CFD

A literatura mostra uma evolução contínua desde os fundamentos variacionais até formulações estabilizadas para escoamentos com convecção dominante. Trabalhos de Galerkin, Hrennikoff e Courant consolidaram a base matemática para aproximações por partes [22, 23, 24]. Em seguida, contribuições de Turner e Clough estruturaram o MEF moderno aplicado à engenharia [25, 26].

No contexto de transporte convectivo e Navier-Stokes, formulações Petrov-Galerkin/SUPG reduziram oscilações numéricas e ampliaram a faixa de estabilidade dos métodos [6, 27, 28, 29]. Benchmarks clássicos de cavidade e degrau regressivo permanecem referências para validação de códigos [30, 31, 32, 33, 34].

Também se destaca a relevância de ferramentas de geração de malha na qualidade da solução, com destaque para o Gmsh em fluxos de trabalho acadêmicos e de pesquisa [35].

2.5 Trabalhos Correlatos (Ênfase em TCCs Semelhantes)

Em trabalhos de graduação recentes, observa-se forte convergência para aplicações de condução de calor e CFD com implementação numérica em Python. Na área térmica, destacam-se estudos sobre discos de freio, dissipadores e componentes eletrônicos heterogêneos [36, 37, 38, 39, 40, 41, 42, 43]. Esses trabalhos reforçam a importância de malhas adequadas, tratamento consistente de contornos e validação por casos de referência.

Na linha de validação metodológica, a comparação entre solução analítica, MDF e MEF foi explorada por Araujo [44], fornecendo evidências de convergência e comportamento de erro em diferentes discretizações. Em aplicações de fluidos e transferência de calor conjugada, também há contribuições relevantes com foco em escoamento para arrefecimento de eletrônicos e análise de campos de velocidade/vorticidade [45, 46, 47, 48, 49].

No contexto local mais recente, trabalhos como os de Gomes [50] e Oliveira [43] reforçam a continuidade da linha de pesquisa em transferência de calor com viés variacional/numérico e MEF tridimensional.

2.6 Síntese da Revisão e Lacuna Atacada

A revisão indica que: (i) há base teórica consolidada para problemas térmicos e de escoamento; (ii) MDF e MEF são métodos complementares, com o MEF mais adequado para geometrias complexas; e (iii) existe demanda por ferramentas didáticas acessíveis para implementação e validação desses métodos.

Nesse cenário, o *MinervaMesh* se posiciona ao integrar formulações clássicas de engenharia numérica em uma plataforma web, com foco em acessibilidade, reprodutibilidade e uso educacional, apoiando-se em bibliotecas científicas amplamente adotadas [5, 4].

Capítulo 3

Fundamentação Teórica

3.1 Introdução ao Capítulo

Este capítulo apresenta a formulação matemática e os métodos numéricos que fundamentam as análises realizadas no *MinervaMesh*. O objetivo central é fornecer uma base sólida, típica da Engenharia Mecânica, para a conversão dos modelos físicos contínuos em sistemas algébricos discretos resolvidos computacionalmente.

Inicialmente, revisam-se as equações diferenciais parciais (EDPs) governantes da condução de calor térmica e do escoamento de fluidos de forma conceitual. Em seguida, após uma breve fundamentação do Método das Diferenças Finitas (MDF), o capítulo se aprofunda extensamente no Método dos Elementos Finitos (MEF). O desenvolvimento abrange desde a formulação de Galerkin, passando pela discretização com elementos triangulares de primeira ordem (P1), até a consolidação da formulação em Vorticidade-Corrente adotada para a dinâmica de fluidos.

3.2 Equações Governantes

A modelagem térmica e fluidodinâmica sustenta-se em princípios de conservação: massa, quantidade de movimento e energia. Estas leis físicas, contínuas no espaço e no tempo, são expressas por EDPs que determinam o comportamento termofluidodinâmico do meio contínuo abordado.

3.2.1 Condução e Convecção Térmica

A transferência de calor em meios sólidos contínuos é regida pela equação da difusão de calor (Lei de Fourier acoplada à conservação da energia térmica). Para um meio estacionário, a distribuição de temperatura $T(\mathbf{x}, t)$ é ditada por [9]:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + q''' \quad (3.1)$$

onde ρ é a massa específica, c_p é o calor específico a pressão constante, k é a condutividade térmica do material, e q''' representa a taxa volumétrica de geração de energia térmica. Para regimes permanentes e propriedades constantes, a Equação 3.1 se simplifica na clássica equação de Poisson (ou Laplace, se $q''' = 0$).

Quando existe escoamento (*convecção*), adiciona-se à equação o transporte advectivo promovido pelo campo de velocidades $\mathbf{v}$:

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = \nabla \cdot (k \nabla T) + q''' \quad (3.2)$$

Esta é a equação de convecção-difusão, paradigma central para o estudo da transferência de calor conjugada ou escoamentos não-isotérmicos.

3.2.2 Escoamento de Fluidos (Navier-Stokes)

O escoamento de fluidos incompressíveis newtonianos é regido pela restrição de continuidade (conservação de massa) e pelas Equações de Navier-Stokes (conservação do momento linear) [14]:

$$\nabla \cdot \mathbf{v} = 0 \quad (3.3)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}_b \quad (3.4)$$

onde p representa o campo de pressão estática, μ a viscosidade dinâmica do fluido e $\mathbf{f}_b$ eventuais forças de campo (como as forças de empuxo gravitacional). Esta formulação em variáveis primitivas (velocidade-pressão) apresenta consideráveis desafios numéricos devido ao acoplamento embutido pela restrição de continuidade [1]. Consequentemente, abordagens numéricas como a Vorticidade-Corrente são frequentemente adotadas em 2D para contornar a incógnita da pressão diretamente.

3.3 Transição Discreta: O Método das Diferenças Finitas (MDF)

Antes do extensivo emprego de formulações variacionais associadas a malhas arbitrárias, a conversão de EDPs a um sistema matricial utilizou maciçamente as propriedades da expansão da série de Taylor. O Método das Diferenças Finitas (MDF) consiste na substituição direta dos operadores diferenciais exatos pelas respectivas aproximações algébricas discretizadas em nós de uma malha tipicamente ortogonal e estruturada [16].

Considere a representação da derivada parcial aproximada pelo método da diferença adiantada e atrasada para aproximações de derivadas espaciais e temporais:

$$\left. \frac{\partial u}{\partial x} \right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x} \quad (\text{Diferença Central}) \quad (3.5)$$

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (3.6)$$

Embora de inegável valor histórico e de extrema eficácia para discretizações explícitas no tempo e problemas de topologia regular simples, o MDF apresenta entraves práticos quando restrições de domínios geométricos altamente complexos exigem representações intrincadas de fronteiras, resultando em interpolações estérreo ou *stair-step* dos contornos físicos do domínio fluido ou de sólidos.

Para dirimir essa dependência geométrica e preservar maior rigor ao tratamento das condições de contorno de von Neumann ou mistas na fronteira, o presente trabalho avança, a seguir, na formulação integral provida pelo Método dos Elementos Finitos (MEF).

3.4 O Método dos Elementos Finitos (MEF)

O Método dos Elementos Finitos constitui a abordagem numérica central adotada neste trabalho. Diferentemente do MDF, o MEF parte de uma formulação integral (variacional ou de resíduos ponderados) da EDP, permitindo o uso de malhas não estruturadas que se adaptam naturalmente a geometrias complexas [18, 8].

Esta seção desenvolve a formulação desde a forma forte até a obtenção do sistema algébrico global.

3.4.1 Da Forma Forte à Forma Fraca

Considere, para fins de generalidade, uma EDP elíptica em um domínio $\Omega \subset \mathbb{R}^d$ com fronteira $\Gamma = \partial\Omega$. De maneira genérica, a *forma forte* do problema visa encontrar $u(\mathbf{x})$ tal que:

$$\mathcal{L}(u) = f \quad \text{em } \Omega \quad (3.7)$$

sujeita às condições de contorno:

$$u = \bar{u} \quad \text{em } \Gamma_D \quad (\text{Dirichlet}) \quad (3.8)$$

$$k \frac{\partial u}{\partial n} = \bar{q} \quad \text{em } \Gamma_N \quad (\text{Neumann}) \quad (3.9)$$

onde $\mathcal{L}$ é um operador diferencial (por exemplo, $-\nabla \cdot (k \nabla(\cdot))$ para condução), f é o termo fonte, $\bar{u}$ é o valor prescrito na fronteira de Dirichlet Γ_D , $\bar{q}$ é o fluxo prescrito na fronteira de Neumann Γ_N , e $\Gamma_D \cup \Gamma_N = \Gamma$.

A *forma fraca* (ou variacional) é obtida multiplicando-se a Equação 3.7 por uma *função-teste* (ou função de ponderação) $w \in \mathcal{V}_0$, pertencente a um espaço funcional adequado, e integrando-se sobre o domínio Ω :

$$\int_{\Omega} w \mathcal{L}(u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.10)$$

A vantagem fundamental desta operação reside na possibilidade de aplicar o Teorema de Green (integração por partes generalizada), transferindo uma ordem de derivação do operador diferencial $\mathcal{L}$ para a função-teste w . Considere o caso concreto da equação de Poisson estacionária ($-\nabla \cdot (k \nabla u) = f$). Multiplicando por w e integrando:

$$-\int_{\Omega} w \nabla \cdot (k \nabla u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.11)$$

Aplicando o Teorema de Green ao lado esquerdo:

$$\int_{\Omega} k \nabla w \cdot \nabla u d\Omega - \int_{\Gamma_N} w k \frac{\partial u}{\partial n} d\Gamma = \int_{\Omega} w f d\Omega \quad (3.12)$$

Substituindo a condição de Neumann (Eq. 3.9) no termo de contorno, obtém-se a *forma fraca*: encontrar $u \in \mathcal{V}$ tal que, para todo $w \in \mathcal{V}_0$:

$$\boxed{\int_{\Omega} k \nabla w \cdot \nabla u \, d\Omega = \int_{\Omega} w f \, d\Omega + \int_{\Gamma_N} w \bar{q} \, d\Gamma} \quad (3.13)$$

Esta expressão é de importância central: ela reduz os requisitos de regularidade sobre u (de C^2 para H^1), incorpora naturalmente as condições de Neumann no funcional, e constitui a base sobre a qual o MEF opera.

3.4.2 O Método de Galerkin

O método de Galerkin consiste em restringir o problema variacional contínuo (Eq. 3.13) a um subespaço de dimensão finita $\mathcal{V}^h \subset \mathcal{V}$, parametrizado por uma malha de elementos finitos. A solução aproximada u^h é expressa como uma combinação linear de N funções de base $\{N_j\}_{j=1}^N$:

$$u^h(\mathbf{x}) = \sum_{j=1}^N U_j N_j(\mathbf{x}) \quad (3.14)$$

onde U_j são os coeficientes nodais (incógnitas do sistema discreto) e $N_j(\mathbf{x})$ são as funções de forma associadas a cada nó j da malha.

A escolha fundamental do método de Galerkin é adotar as *mesmas* funções de base como funções-teste: $w^h = N_i$, $i = 1, \dots, N$. Substituindo na forma fraca (Eq. 3.13):

$$\sum_{j=1}^N U_j \int_{\Omega} k \nabla N_i \cdot \nabla N_j \, d\Omega = \int_{\Omega} N_i f \, d\Omega + \int_{\Gamma_N} N_i \bar{q} \, d\Gamma \quad \forall i = 1, \dots, N \quad (3.15)$$

Esta expressão define um sistema linear $\mathbf{KU} = \mathbf{F}$, onde:

$$K_{ij} = \int_{\Omega} k \nabla N_i \cdot \nabla N_j \, d\Omega \quad (\text{Matriz de Rigidez}) \quad (3.16)$$

$$F_i = \int_{\Omega} N_i f \, d\Omega + \int_{\Gamma_N} N_i \bar{q} \, d\Gamma \quad (\text{Vetor de Carga}) \quad (3.17)$$

A resolução deste sistema algébrico fornece os coeficientes nodais $\mathbf{U}$, a partir dos quais o campo aproximado u^h é reconstituído via Equação 3.14. A qualidade da

aproximação depende diretamente da riqueza do espaço $\mathcal{V}^h$, controlada pelo refinamento da malha e pela ordem polinomial das funções de base [7].

De maneira compacta, o procedimento de Galerkin converte o problema contínuo em:

$$\boxed{\mathbf{K}\mathbf{U} = \mathbf{F}} \quad (3.18)$$

A construção efetiva das matrizes $\mathbf{K}$ e do vetor $\mathbf{F}$ depende da escolha dos elementos e das funções de forma, desenvolvida na seção seguinte.

3.4.3 Elementos Triangulares Lineares (P1)

A discretização do domínio Ω é realizada por uma malha de N_e elementos triangulares, cujos vértices coincidem com os N nós globais da malha. Cada elemento e possui três nós locais $(1, 2, 3)$, com coordenadas (x_i, y_i) , $i = 1, 2, 3$.

As funções de forma lineares N_i^e para o elemento triangular P1 são definidas de modo que $N_i^e(\mathbf{x}_j) = \delta_{ij}$ (propriedade de partição da unidade nodal). Em coordenadas cartesianas, estas funções são expressas por [8]:

$$N_i^e(x, y) = \frac{1}{2A^e}(a_i + b_i x + c_i y), \quad i = 1, 2, 3 \quad (3.19)$$

onde A^e é a área do elemento triangular e os coeficientes são dados por permutação cíclica dos índices dos vértices:

$$a_i = x_j y_k - x_k y_j, \quad b_i = y_j - y_k, \quad c_i = x_k - x_j \quad (3.20)$$

com (i, j, k) em ordem cíclica $(1, 2, 3)$, $(2, 3, 1)$, $(3, 1, 2)$. A área do elemento é calculada por:

$$A^e = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)| \quad (3.21)$$

Os gradientes das funções de forma são constantes dentro de cada elemento P1:

$$\nabla N_i^e = \frac{1}{2A^e} \begin{pmatrix} b_i \\ c_i \end{pmatrix} \quad (3.22)$$

Esta propriedade simplifica consideravelmente as integrais elementares, uma vez que os integrandos envolvendo $\nabla N_i \cdot \nabla N_j$ tornam-se constantes no interior do elemento.

3.4.4 Matrizes Elementares

A partir das funções de forma P1, as integrais da formulação de Galerkin (Eqs. 3.16 e 3.17) são calculadas elemento a elemento e, em seguida, assembradas no sistema global.

3.4.4.1 Matriz de Rigidez Elementar

Para o problema de difusão com condutividade k constante por elemento, a matriz de rigidez elementar $\mathbf{K}^e$ de dimensão 3×3 é:

$$K_{ij}^e = \int_{\Omega^e} k \nabla N_i^e \cdot \nabla N_j^e d\Omega = \frac{k}{4A^e} (b_i b_j + c_i c_j) \quad (3.23)$$

3.4.4.2 Matriz de Massa Elementar

Em problemas transientes, a discretização temporal introduz um termo de acumulação que requer a *matriz de massa*. Para elementos P1, a matriz de massa consistente é [18]:

$$M_{ij}^e = \int_{\Omega^e} \rho c_p N_i^e N_j^e d\Omega = \frac{\rho c_p A^e}{12} \begin{cases} 2 & \text{se } i = j \\ 1 & \text{se } i \neq j \end{cases} \quad (3.24)$$

Alternativamente, a *matriz de massa concentrada* (*lumped*) distribui a massa uniformemente nos nós, resultando em uma matriz diagonal:

$$M_{ij}^{e,\text{lump}} = \frac{\rho c_p A^e}{3} \delta_{ij} \quad (3.25)$$

3.4.4.3 Vetor de Carga Elementar

Para um termo fonte f uniforme no elemento, o vetor de carga elementar resulta em:

$$F_i^e = \int_{\Omega^e} N_i^e f d\Omega = \frac{f A^e}{3} \quad (3.26)$$

3.4.5 Montagem Global

A construção do sistema global $\mathbf{KU} = \mathbf{F}$ é realizada pelo procedimento de *assembly*, que consiste em acumular as contribuições de cada elemento nas posições globais correspondentes. Para cada elemento e , a conectividade local-global é definida por um mapa de índices $\text{IEN}(a, e) \rightarrow I$, que associa o nó local a do elemento e ao nó global I .

A montagem é expressa como:

$$K_{IJ} = \sum_{e=1}^{N_e} K_{\text{IEN}(a,e), \text{IEN}(b,e)}^e, \quad F_I = \sum_{e=1}^{N_e} F_{\text{IEN}(a,e)}^e \quad (3.27)$$

Após a montagem, as condições de contorno de Dirichlet são impostas modificando-se as linhas correspondentes do sistema. A resolução do sistema linear resultante fornece os valores nodais $\mathbf{U}$.

3.5 Formulação Vorticidade-Corrente

A resolução direta das equações de Navier-Stokes em variáveis primitivas (velocidade-pressão) apresenta dificuldades numéricas associadas ao acoplamento pressão-velocidade e à satisfação da restrição de continuidade (Eq. 3.3). Em problemas bidimensionais, a formulação em *Vorticidade-Corrente* (ω - ψ) oferece uma alternativa eficaz, eliminando o campo de pressão das incógnitas e reduzindo o sistema a duas equações escalares [1, 2].

3.5.1 Definições

Para um escoamento bidimensional, define-se a *função corrente* $\psi(x, y, t)$ tal que o campo de velocidades satisfaça identicamente a equação da continuidade:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (3.28)$$

A *vorticidade*, componente escalar no plano 2D, é definida como o rotacional do campo de velocidades:

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (3.29)$$

Substituindo as definições da função corrente na expressão da vorticidade, obtém-se a *equação de Poisson para a função corrente*:

$$\nabla^2 \psi = -\omega \quad (3.30)$$

3.5.2 Equação de Transporte da Vorticidade

Aplicando o rotacional às equações de Navier-Stokes (Eq. 3.4), o gradiente de pressão é eliminado e obtém-se a equação de transporte da vorticidade:

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \nu \nabla^2 \omega \quad (3.31)$$

onde $\nu = \mu/\rho$ é a viscosidade cinemática. Esta equação possui a estrutura clássica de convecção-difusão, onde o campo de velocidades (u, v) transporta a vorticidade ω enquanto a difusão viscosa a dissipa.

3.5.3 Sistema Acoplado e Solução Iterativa

O sistema ω - ψ é resolvido iterativamente. A cada passo de tempo (ou iteração estacionária), o procedimento consiste em:

1. Resolver a equação de Poisson (Eq. 3.30) para ψ , dado ω .
2. Calcular o campo de velocidades (u, v) a partir de ψ (Eq. 3.28).
3. Resolver a equação de transporte (Eq. 3.31) para ω , dado (u, v) .
4. Verificar a convergência e repetir, se necessário.

3.5.4 Matriz de Convecção e Estabilização SUPG

A discretização via MEF do operador convectivo $\mathbf{v} \cdot \nabla \phi$ conduz à *matriz de convecção* elementar, de dimensão 3×3 para elementos P1:

$$C_{ij}^e = \int_{\Omega^e} N_i (\mathbf{v} \cdot \nabla N_j) d\Omega = \frac{A^e}{3} \left(\frac{u_e b_j + v_e c_j}{2A^e} \right) \quad (3.32)$$

onde u_e e v_e são as componentes de velocidade avaliadas no centroide do elemento.

Quando o campo de velocidades é intenso em relação à difusão (número de Péclet elevado), a formulação de Galerkin padrão produz oscilações espúrias. Para contornar essa limitação, emprega-se a técnica *Streamline Upwind Petrov-Galerkin* (SUPG), que adiciona uma difusão artificial orientada na direção do escoamento [6].

Na formulação SUPG, a função-teste é modificada para $w_i = N_i + \tau_{\text{SUPG}} \mathbf{v} \cdot \nabla N_i$, com o parâmetro de estabilização:

$$\tau_{\text{SUPG}} = \frac{h_e}{2\|\mathbf{v}\|} \left(\coth \text{Pe}_e - \frac{1}{\text{Pe}_e} \right) \quad (3.33)$$

onde $\text{Pe}_e = \|\mathbf{v}\| h_e / (2\alpha)$ é o número de Péclet elementar.

No *MinervaMesh*, a estabilização SUPG é empregada no caso unidimensional de convecção-difusão, onde o efeito do número de Péclet é mais crítico. No solver bidimensional de Navier-Stokes, a convecção da vorticidade utiliza a formulação de Galerkin padrão (Eq. 3.32), sendo a estabilidade controlada pela escolha adequada do passo de tempo e do número de Reynolds.

3.6 Integração Temporal

Para problemas transientes, a discretização temporal converte a EDP semi-discreta (após a discretização espacial por MEF) em um sistema algébrico a cada intervalo de tempo Δt . No caso geral, o sistema semidiscreto assume a forma:

$$\mathbf{M} \frac{d\mathbf{U}}{dt} + \mathbf{K}\mathbf{U} = \mathbf{F}(t) \quad (3.34)$$

3.6.1 Método θ Generalizado

A família de métodos θ parametriza a ponderação temporal entre os instantes t^n e t^{n+1} :

$$\mathbf{M} \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t} + \mathbf{K} [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] = \theta \mathbf{F}^{n+1} + (1 - \theta) \mathbf{F}^n \quad (3.35)$$

Os casos particulares mais relevantes são:

- $\theta = 0$: Euler explícito (condicionalmente estável);
- $\theta = 1/2$: Crank-Nicolson (incondicionalmente estável, segunda ordem);
- $\theta = 1$: Euler implícito (incondicionalmente estável, primeira ordem).

3.6.2 Esquema Semi-Implícito Adotado

No solver de Navier-Stokes do *MinervaMesh*, a equação de transporte da vorticidade (Eq. 3.31) é discretizada com um esquema *semi-implícito*: o termo difusivo é tratado implicitamente ($\theta = 1$), enquanto o termo convectivo é avaliado explicitamente no instante t^n . Isso resulta no sistema:

$$\left(\mathbf{M} + \frac{\Delta t}{\text{Re}} \mathbf{K} \right) \boldsymbol{\omega}^{n+1} = \mathbf{M} \boldsymbol{\omega}^n - \Delta t \mathbf{C}^n \boldsymbol{\omega}^n \quad (3.36)$$

onde $\mathbf{C}^n$ é a matriz de convecção assembrada a partir do campo de velocidades no instante atual e $\text{Re} = \rho UL/\mu$ é o número de Reynolds. O tratamento implícito da difusão garante estabilidade incondicional para esse operador, enquanto a convecção explícita impor restrições sobre Δt associadas ao número de Courant.

Para problemas puramente difusivos (condução transiente), utiliza-se o método θ generalizado (Eq. 3.35), permitindo ao usuário selecionar entre Euler e Crank-Nicolson conforme o compromisso desejado entre precisão e estabilidade.

3.6.3 Condição de Contorno de Vorticidade: Fórmula de Thom

Na formulação ω - ψ , a vorticidade nas paredes sólidas não é conhecida *a priori* e deve ser calculada indiretamente a partir do campo de função corrente. A fórmula de Thom [1] fornece essa condição de contorno, baseando-se em uma expansão de Taylor de ψ na direção normal à parede:

$$\omega_w = -\frac{2(\psi_{\text{nb}} - \psi_w - h u_{\text{tan}})}{h^2} \quad (3.37)$$

onde ψ_w e ψ_{nb} são os valores da função corrente na parede e no nó vizinho mais próximo na direção normal, respectivamente; h é a distância entre esses nós; e u_{tan} é

a velocidade tangencial da parede ($u_{\text{tan}} = 0$ para paredes estacionárias e $u_{\text{tan}} = U_{\text{lid}}$ para paredes móveis, como no problema da cavidade).

3.7 Síntese do Capítulo

Este capítulo estabeleceu o arcabouço matemático completo empregado pelo *MinervaMesh*. A formulação parte das EDPs governantes para fenômenos térmicos e fluidodinâmicos, transita pela introdução conceitual do MDF e se aprofunda no MEF via Galerkin. A discretização com elementos triangulares P1 permite a construção explícita das matrizes elementares de rigidez, massa e convecção, enquanto a formulação Vorticidade-Corrente viabiliza a solução de escoamentos 2D sem a necessidade de resolver diretamente o campo de pressão. A integração temporal semi-implícita e a fórmula de Thom para condições de contorno de vorticidade completam o ferramental necessário para a simulação dos casos apresentados no Capítulo 5.

Capítulo 4

Desenvolvimento e Arquitetura

4.1 Visão Geral da Plataforma

O *MinervaMesh* é uma plataforma de simulação numérica projetada para operar integralmente no navegador web, sem dependência de servidores de processamento remoto nem instalação de software por parte do usuário. A plataforma reúne implementações dos métodos numéricos apresentados no Capítulo 3 em uma interface interativa que permite a parametrização do problema, a execução da simulação e a visualização dos resultados em uma única página.

A motivação central do projeto reside na eliminação das barreiras de acesso associadas às ferramentas de simulação tradicionais. Ambientes como ANSYS, COMSOL Multiphysics ou mesmo distribuições locais de Python científico exigem licenciamento, infraestrutura computacional dedicada e configuração de ambiente — requisitos que frequentemente inviabilizam o uso autônomo em disciplinas de graduação, monitorias e estudos independentes [3]. O *MinervaMesh* contorna essas limitações ao executar todo o processamento no dispositivo do próprio usuário, utilizando o navegador como ambiente de execução.

A plataforma abrange três domínios da Engenharia Mecânica:

- **Transferência de Calor:** condução estacionária e transiente, convecção-difusão com estabilização SUPG, transferência de calor 2D;

- **Mecânica dos Fluidos:** escoamento potencial, Navier-Stokes 2D via formulação Vorticidade-Corrente;
- **Mecânica Estrutural:** flexão de vigas (Euler-Bernoulli) e análise modal de vibração.

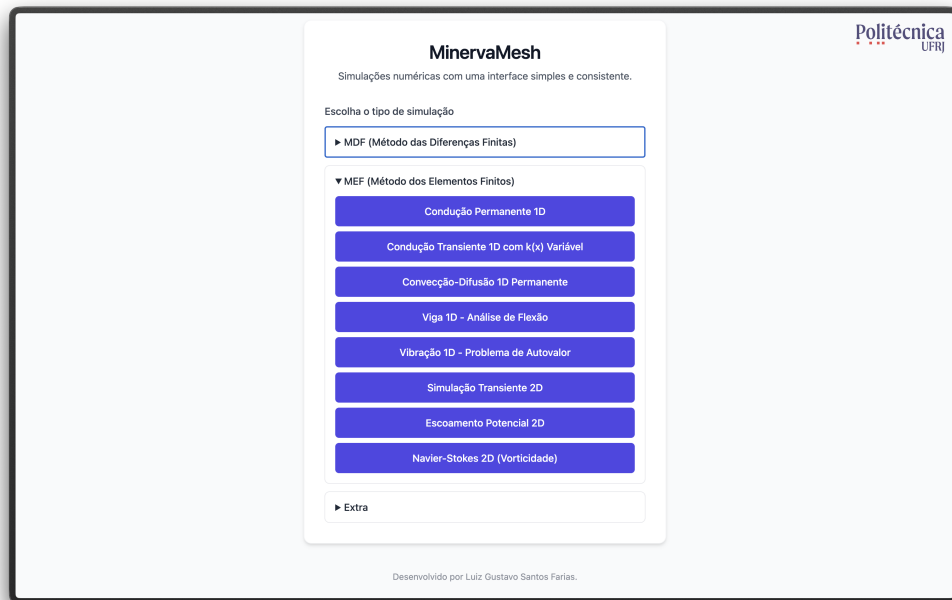


Figura 4.1: Tela inicial do *MinervaMesh* com o menu de simulações disponíveis.

4.2 Justificativa Tecnológica

4.2.1 PyScript e WebAssembly

A escolha do PyScript como runtime da plataforma fundamenta-se em três critérios: portabilidade, familiaridade da linguagem e manutenibilidade.

O PyScript é um *framework* de código aberto que permite a execução de programas Python diretamente no navegador, apoiando-se no Pyodide — uma compilação do interpretador CPython para WebAssembly (Wasm) [4]. O WebAssembly é um formato binário padronizado pelo W3C que executa código de baixo nível em velocidade próxima à nativa, dentro da *sandbox* de segurança do navegador. A cadeia de execução é:

1. O navegador carrega a página HTML contendo a tag `<script type="py">`.
2. O *runtime* do PyScript inicializa o Pyodide, que instancia uma máquina virtual CPython compilada para WebAssembly.
3. As dependências declaradas em `config.toml` (NumPy, SciPy, Matplotlib) são carregadas como pacotes pré-compilados para Wasm.
4. O código Python da simulação é interpretado pelo Pyodide, com acesso direto ao DOM do navegador para leitura de parâmetros e renderização de resultados.

A escolha de Python como linguagem de implementação é estratégica para o contexto educacional: trata-se da linguagem mais utilizada em disciplinas de métodos numéricos na Engenharia, com sintaxe próxima à notação matemática e ecossistema científico consolidado (NumPy para álgebra linear, SciPy para solvers esparsos, Matplotlib para visualização) [5]. Ao manter a implementação em Python puro, o *MinervaMesh* permite que alunos inspecionem, compreendam e modifiquem o código das simulações — funcionalidade explicitamente disponível através do botão “Ver Código” presente em cada simulação.

4.2.2 Computação *Client-Side*

A arquitetura *client-side* implica que todo o processamento numérico ocorre no dispositivo do usuário. Esta decisão tem impactos diretos em quatro dimensões:

1. **Custo de infraestrutura:** o *MinervaMesh* é um site estático. Pode ser hospedado gratuitamente em plataformas como GitHub Pages ou Netlify, sem necessidade de servidor *backend*, banco de dados ou infraestrutura de *cloud computing*. O custo operacional é efetivamente zero.
2. **Escalabilidade:** não existe gargalo de servidor central. Cada usuário utiliza o poder computacional de sua própria máquina, eliminando filas de processamento e limites de usuários simultâneos.
3. **Privacidade:** nenhum dado de simulação transita pela rede. Parâmetros, malhas e resultados permanecem integralmente no dispositivo do usuário.

4. **Disponibilidade:** após o carregamento inicial dos *assets*, a plataforma pode operar sem conexão com a internet, viabilizando o uso em ambientes com conectividade limitada.

4.2.3 Comparação com Alternativas

A Tabela 4.1 posiciona o *MinervaMesh* frente às alternativas mais comuns no contexto de ensino de simulação numérica.

Tabela 4.1: Comparação entre plataformas de simulação numérica para ensino.

Critério	MinervaMesh	MATLAB	Jupyter	COMSOL
Custo de licença	Gratuito	Pago	Gratuito	Pago
Instalação necessária	Não	Sim	Sim	Sim
Execução no navegador	Sim	Não	Parcial	Não
Código-fonte acessível	Sim	Parcial	Sim	Não
Infraestrutura de servidor	Não	Não	Sim	Sim

A principal limitação da abordagem *client-side* reside na capacidade computacional: problemas com malhas muito refinadas (dezenas de milhares de nós) podem apresentar tempo de execução elevado, uma vez que o desempenho do WebAssembly, embora próximo ao nativo, ainda é inferior ao de implementações compiladas em C/Fortran. Para os casos de interesse deste trabalho, envolvendo malhas de centenas a poucos milhares de nós, essa limitação não se mostrou restritiva.

4.3 Arquitetura de Software

4.3.1 Organização de Diretórios

O repositório do *MinervaMesh* segue uma estrutura modular, onde cada simulação é autocontida em seu próprio diretório. A organização geral é apresentada a seguir:

```
minervamesh/
|-- index.html          # Menu principal
|-- assets/             # Logos e imagens globais
```



```

|-- simulations/
|   |-- mdf/                                # Diferencas Finitas
|   |   |-- conducao-permanente-barra1d-geracao/
|   |   +--- equacao-onda/
|   |-- mef/                                # Elementos Finitos
|   |   |-- conducao-permanente-barra1d-mef/
|   |   |-- calor-transiente-1d/
|   |   |-- conveccao-difusao-1d/
|   |   |-- viga-1d-mef/
|   |   |-- vibracao-1d-mef/
|   |   |-- transcalor/
|   |   +--- escoamento-fluido-2d/
|   |       |-- common.py                # Matrizes MEF
|   |       |-- geometry.py             # Geometrias
|   |       |-- escoamento-potencial/
|   |       +--- navier-stokes/
|   +--- extra/
|       +--- gerador-malha/

```

Cada módulo de simulação segue um padrão uniforme de três arquivos:

- `index.html`: interface do usuário, contendo o formulário de parâmetros, o contêiner de resultados e a carga do PyScript;
- `simulation.py`: implementação da lógica numérica em Python, incluindo montagem de matrizes, solução do sistema e geração de gráficos;
- `config.toml`: declaração das dependências Python necessárias (por exemplo, `numpy`, `scipy`, `matplotlib`).

4.3.2 Fluxo de Execução de uma Simulação

O ciclo de vida de uma simulação no *MinervaMesh* pode ser descrito em cinco etapas:

1. **Carregamento:** o navegador requisita o `index.html` de um servidor HTTP simples. A tag `<script type="py">` instrui o PyScript a inicializar o Pyodide e carregar as dependências declaradas em `config.toml`.
2. **Parametrização:** o usuário configura os parâmetros físicos e numéricos através de campos de entrada na interface.
3. **Execução:** ao clicar em “Rodar”, o código Python é executado pelo Pyodide. As matrizes do MEF são montadas, o sistema linear é resolvido e os gráficos são gerados — tudo no navegador.
4. **Visualização:** os gráficos resultantes são convertidos para formato de imagem codificado em Base64 e inseridos dinamicamente no DOM da página.
5. **Inspeção:** o botão “Ver Código” exibe o código-fonte Python da simulação com *syntax highlighting*, permitindo ao aluno estudar a implementação.

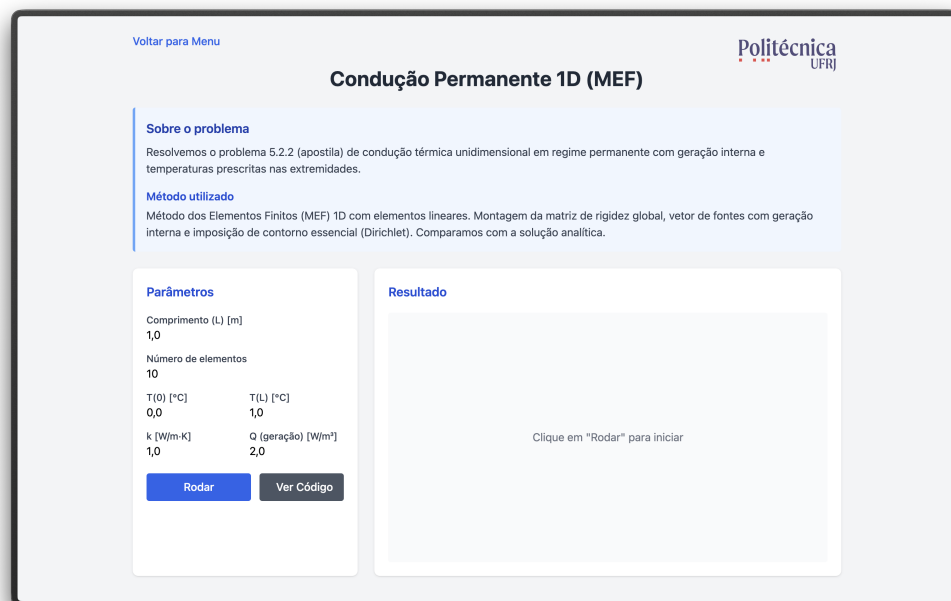


Figura 4.2: Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.

4.4 Módulos de Simulação MEF

Esta seção descreve cada módulo de simulação implementado no *Minerva-Mesh*, detalhando o problema físico resolvido, as decisões de implementação e os resultados obtidos. Os trechos de código apresentados referem-se às funções centrais de cada simulação; as listagens completas encontram-se nos Apêndices.

4.4.1 Condução Permanente 1D

O primeiro módulo resolve o problema de condução térmica unidimensional em regime permanente com geração interna de calor. A equação governante é a forma 1D da equação de Poisson (Eq. 3.1 com $\partial T / \partial t = 0$):

$$-k \frac{d^2 T}{dx^2} = Q, \quad T(0) = T_0, \quad T(L) = T_L \quad (4.1)$$

A discretização por elementos finitos lineares resulta na matriz de rigidez e no vetor de carga elementares (Seção 3.4.4), implementados como:

```
# Elemento linear 1D: ke e be
ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
be = (Q * h / 2.0) * np.array([1.0, 1.0])
```

A montagem percorre os N_{el} elementos, acumulando as contribuições nas posições globais. Após a imposição das condições de Dirichlet (zerando linhas e colunas correspondentes), o sistema $\mathbf{KT} = \mathbf{b}$ é resolvido por `numpy.linalg.solve`. A Figura 4.3 mostra a comparação entre a solução MEF e a solução analítica.



Figura 4.3: Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.

4.4.2 Condução Transiente 1D com $k(x)$ Variável

Este módulo estende o problema anterior para o regime transiente, incorporando a matriz de massa e a integração temporal. A equação governante é:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k(x) \frac{\partial T}{\partial x} \right) \quad (4.2)$$

com condutividade térmica variável $k(x)$, definida como $k_1 = 5,0$ para $x < L/2$ e $k_2 = 1,0$ para $x \geq L/2$.

As matrizes elementares são calculadas por:

```
# Matrizes de elemento 1D transiente
ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])
me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])
```

A integração temporal utiliza o método θ generalizado (Eq. 3.35), permitindo ao usuário selecionar entre Euler explícito, Crank-Nicolson e Euler implícito. A Figura 4.4 ilustra a evolução temporal do campo de temperatura.

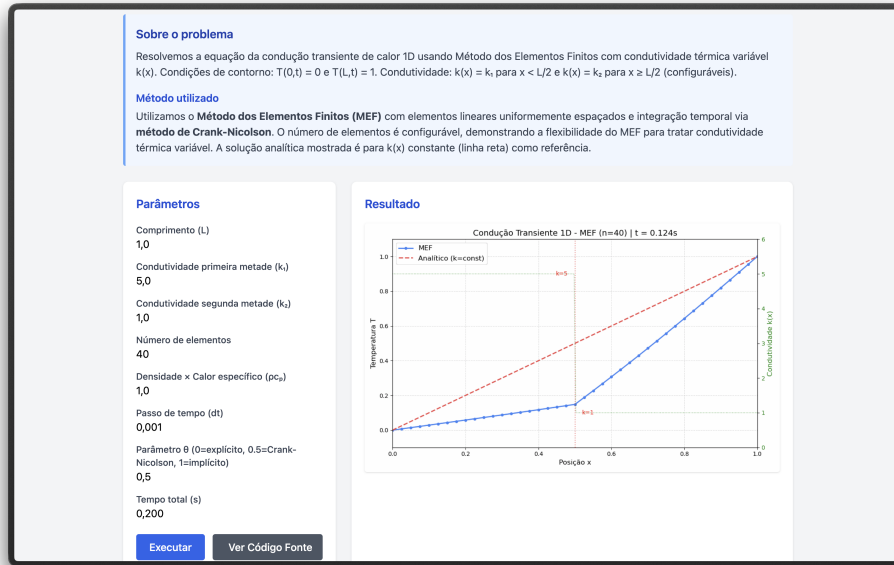


Figura 4.4: Evolução temporal do campo de temperatura com condutividade variável $k(x)$.

4.4.3 Convecção-Difusão 1D com Estabilização SUPG

O módulo de convecção-difusão 1D resolve a equação de advecção-difusão estacionária:

$$-k \frac{d^2 T}{dx^2} + \rho c_p u \frac{dT}{dx} = Q \quad (4.3)$$

A discretização MEF gera três matrizes elementares: difusão, convecção e estabilização SUPG (Seção 3.5.4). O cálculo do parâmetro de estabilização na implementação é:

```
# Número de Péclet local e tau SUPG
Pe_local = (rho_cp * u * h) / (2 * k)
tau = h / (2*abs(u)) * (1/np.tanh(abs(Pe_local))
    - 1/abs(Pe_local))
```

A Figura 4.5 demonstra o efeito da estabilização SUPG em um problema com número de Péclet elevado. Sem estabilização, a formulação de Galerkin padrão produz oscilações espúrias; com SUPG, a solução numérica converge suavemente para a solução analítica.

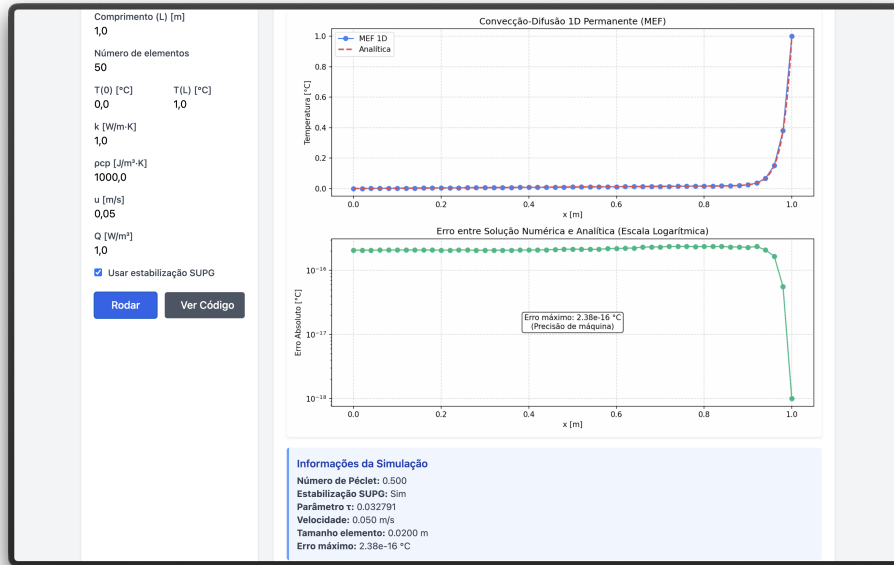


Figura 4.5: Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.

4.4.4 Viga 1D — Análise de Flexão (Euler-Bernoulli)

O módulo de viga 1D implementa elementos finitos de Euler-Bernoulli com dois graus de liberdade por nó: deslocamento vertical w e rotação θ . A equação governante para a flexão estática é:

$$\frac{d^2}{dx^2} \left(EI \frac{d^2 w}{dx^2} \right) = q(x) \quad (4.4)$$

A matriz de rigidez elementar 4×4 é construída com as funções de interpolação cúbicas de Hermite:

```
# Matriz de rigidez do elemento de viga
ke = (EI / h**3) * np.array([
    [12,   6*h,  -12,   6*h ],
    [6*h,  4*h**2, -6*h,  2*h**2],
    [-12, -6*h,   12,  -6*h ],
    [6*h,  2*h**2, -6*h,  4*h**2]
])
```

O módulo suporta quatro condições de contorno (simplesmente apoiada, balanço, engastada-engastada e engastada-apoiada) e cinco tipos de carregamento (uniforme, linear, triangular, senoidal e pontual), além de propriedades materiais variáveis ao longo do comprimento. A Figura 4.6 apresenta a análise completa de uma viga, incluindo deflexão, diagramas de momento fletor e esforço cortante.

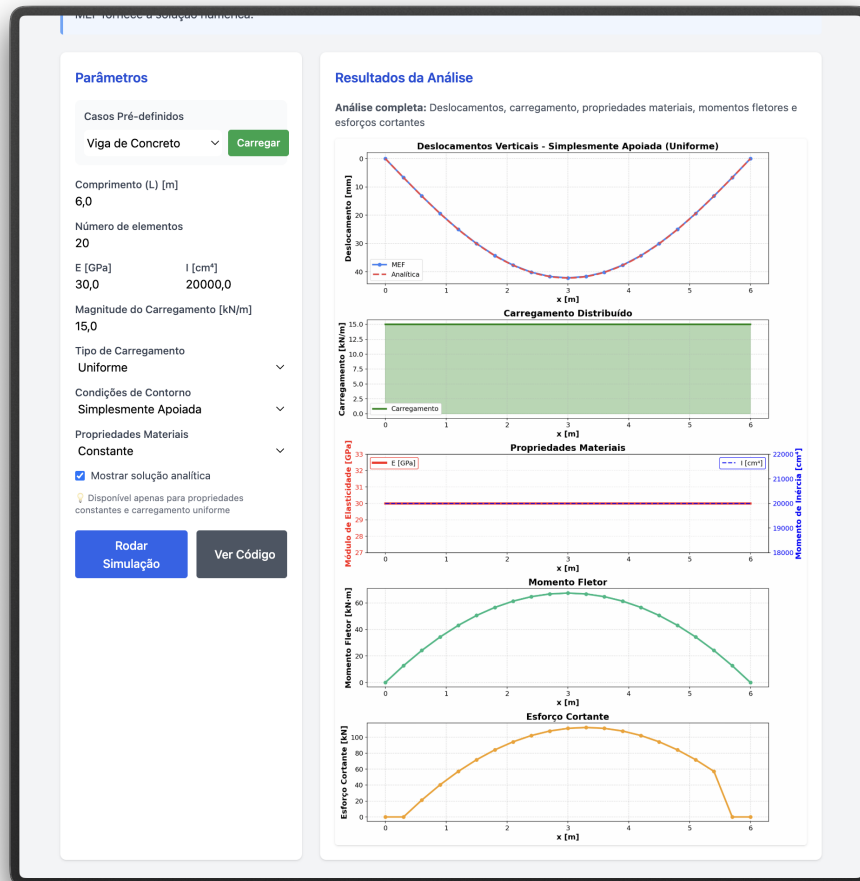


Figura 4.6: Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.

4.4.5 Vibração 1D — Problema de Autovalor

O módulo de vibração resolve o problema de autovalor generalizado $\mathbf{K}\phi = \lambda \mathbf{M}\phi$ para uma barra elástica 1D, onde as frequências naturais são $f_n = \sqrt{\lambda_n}/(2\pi)$ e os autovetores ϕ_n representam os modos de vibração. As matrizes elementares utilizam uma massa concentrada (*lumped*):

```
# Matrizes de rigidez e massa (barra 1D)
ke = (E*A / h) * np.array([[1, -1], [-1, 1]])
me = (rho*A*h / 2) * np.array([[1, 0], [0, 1]])
```

O problema de autovalor é resolvido pela rotina `scipy.linalg.eigh`, que explora a simetria das matrizes para eficiência e estabilidade numérica. A solução fornece n frequências naturais e os correspondentes modos de vibração, comparáveis com as expressões analíticas $f_n = nc/(2L)$ para barras engastadas, onde $c = \sqrt{E/\rho}$ é a velocidade de propagação de ondas. A Figura 4.7 ilustra os cinco primeiros modos.

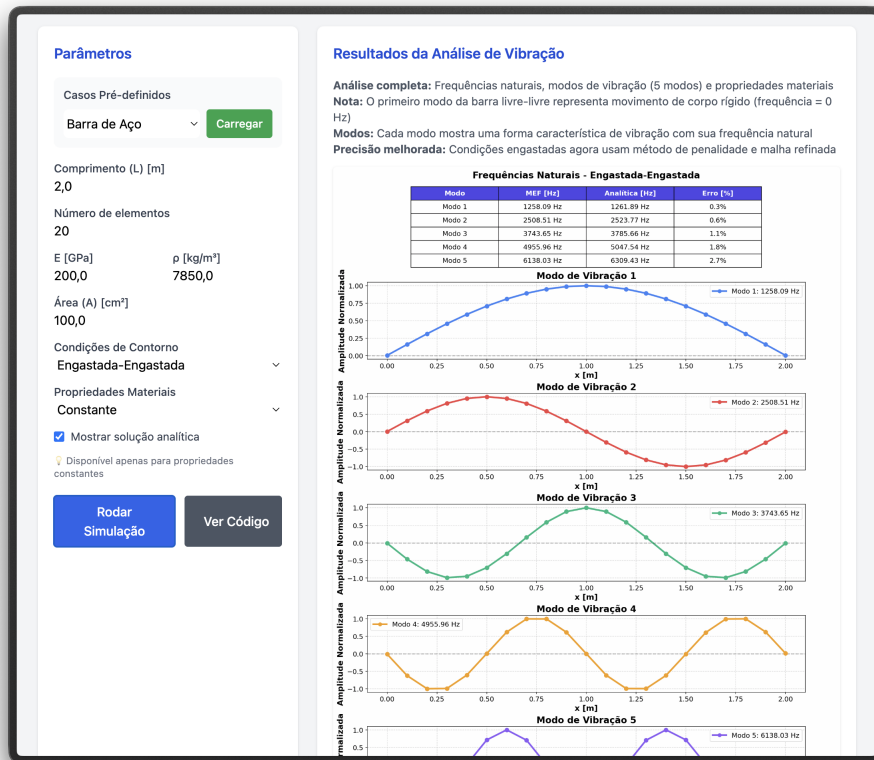


Figura 4.7: Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.

4.4.6 Transferência de Calor 2D

O módulo de transferência de calor bidimensional resolve a equação de condução transiente em uma placa quadrada discretizada com elementos triangulares P1. As

matrizes elementares de rigidez e massa são construídas pela função compartilhada `element_matrices`, que implementa as expressões derivadas na Seção 3.4.4:

```
def element_matrices(coords, k, rho, cv):
    x0, x1, x2 = coords
    area = 0.5 * abs((x1[0]-x0[0])*(x2[1]-x0[1])
                    -(x2[0]-x0[0])*(x1[1]-x0[1]))
    b = np.array([x1[1]-x2[1], x2[1]-x0[1], x0[1]-x1[1]])
    c = np.array([x2[0]-x1[0], x0[0]-x2[0], x1[0]-x0[0]])
    ke = (k/(4*area)) * (np.outer(b,b) + np.outer(c,c))
    me = (rho*cv*area/12) * (np.ones((3,3)) + np.eye(3))
    return ke, me
```

A integração temporal emprega o método de Crank-Nicolson ($\theta = 1/2$), formulado com matrizes esparsas (`scipy.sparse`) para viabilizar a execução no navegador com malhas de até 40×40 nós. A cada passo de tempo, um frame do campo de temperatura é gerado e acumulado para compor uma animação GIF. A Figura 4.8 ilustra um instante da evolução temporal.

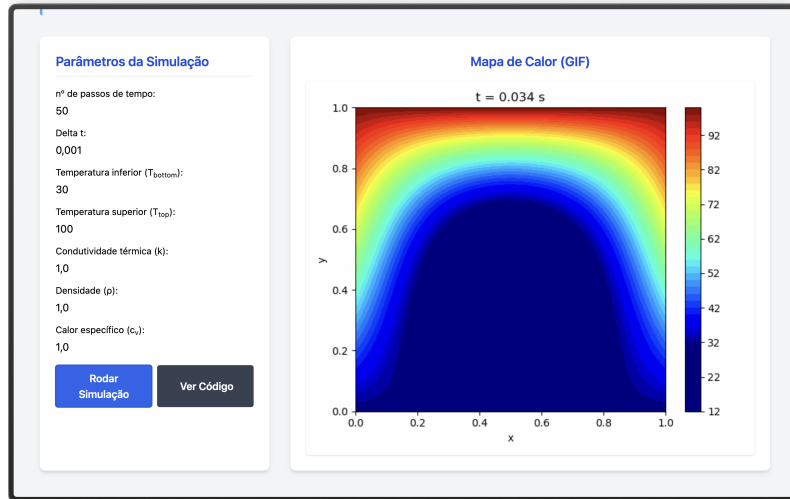


Figura 4.8: Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).

4.4.7 Escoamento Potencial 2D

O módulo de escoamento potencial resolve a equação de Laplace $\nabla^2\psi = 0$ em domínios 2D com obstáculos, determinando o campo de função corrente e, por derivação, o campo de velocidades. A malha triangular é gerada pelo módulo `common.py`, que também assembla as matrizes globais de rigidez e massa.

O módulo `geometry.py` fornece diferentes geometrias de obstáculo (cilindro, retângulo, degrau), sendo os nós do contorno do obstáculo inseridos na malha com a condição $\psi = \text{constante}$. As velocidades são recuperadas por:

$$u = \frac{\partial\psi}{\partial y} \approx \frac{1}{2A^e} \sum_j \psi_j c_j, \quad v = -\frac{\partial\psi}{\partial x} \approx -\frac{1}{2A^e} \sum_j \psi_j b_j \quad (4.5)$$

A Figura 4.9 apresenta as linhas de corrente para escoamento ao redor de um obstáculo.

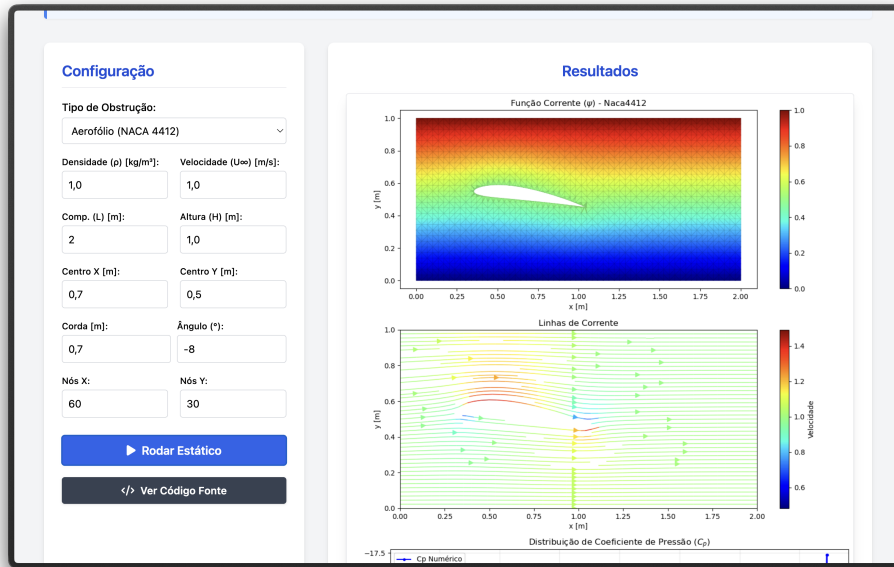


Figura 4.9: Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.

4.4.8 Navier-Stokes 2D (Vorticidade-Corrente)

O módulo mais complexo da plataforma implementa o solver de Navier-Stokes 2D na formulação Vorticidade-Corrente ($\omega-\psi$), conforme descrito na Seção 3.5. O algoritmo segue o procedimento iterativo:

1. Resolver a equação de Poisson $\mathbf{K}\psi = \mathbf{M}\omega$ para ψ ;

2. Atualizar a vorticidade nas paredes sólidas pela fórmula de Thom (Eq. 3.37);
3. Calcular (u, v) a partir de ψ (Eq. 4.5);
4. Assemblar a matriz de convecção $\mathbf{C}^n$ com o campo de velocidades atual;
5. Resolver o transporte da vorticidade pelo esquema semi-implícito (Eq. 3.36).

O solver suporta dois cenários: escoamento em canal com obstáculos (cilindro, retângulo, degrau) e cavidade com tampa móvel (*lid-driven cavity*). No caso da cavidade, todas as paredes possuem $\psi = 0$ e a parede superior impõe velocidade tangencial $u_{\text{lid}} = 1$, conforme a generalização da fórmula de Thom com parede móvel. A Figura 4.10 mostra os campos de ψ , ω e as linhas de corrente para a cavidade a $\text{Re} = 100$.

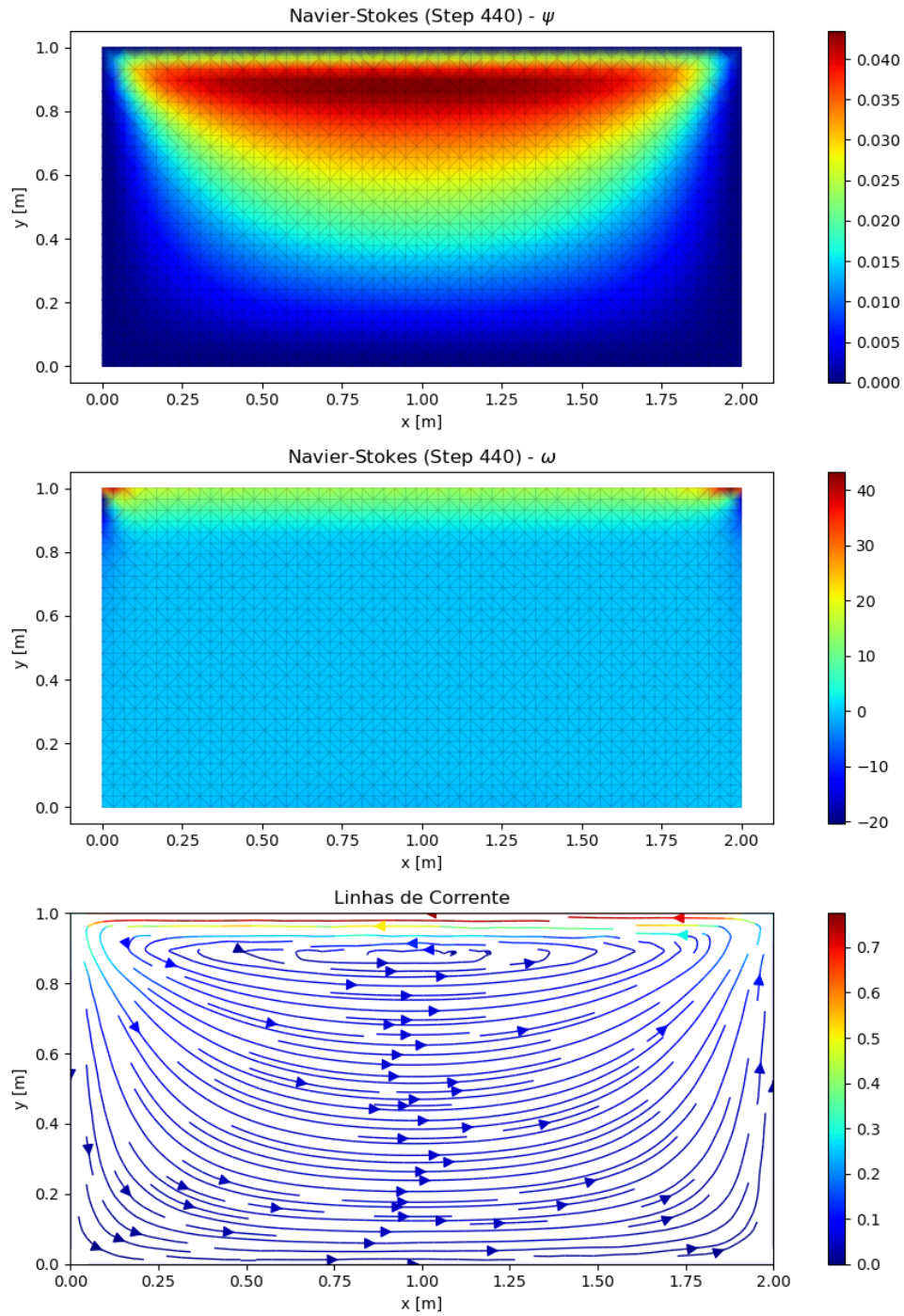


Figura 4.10: Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.

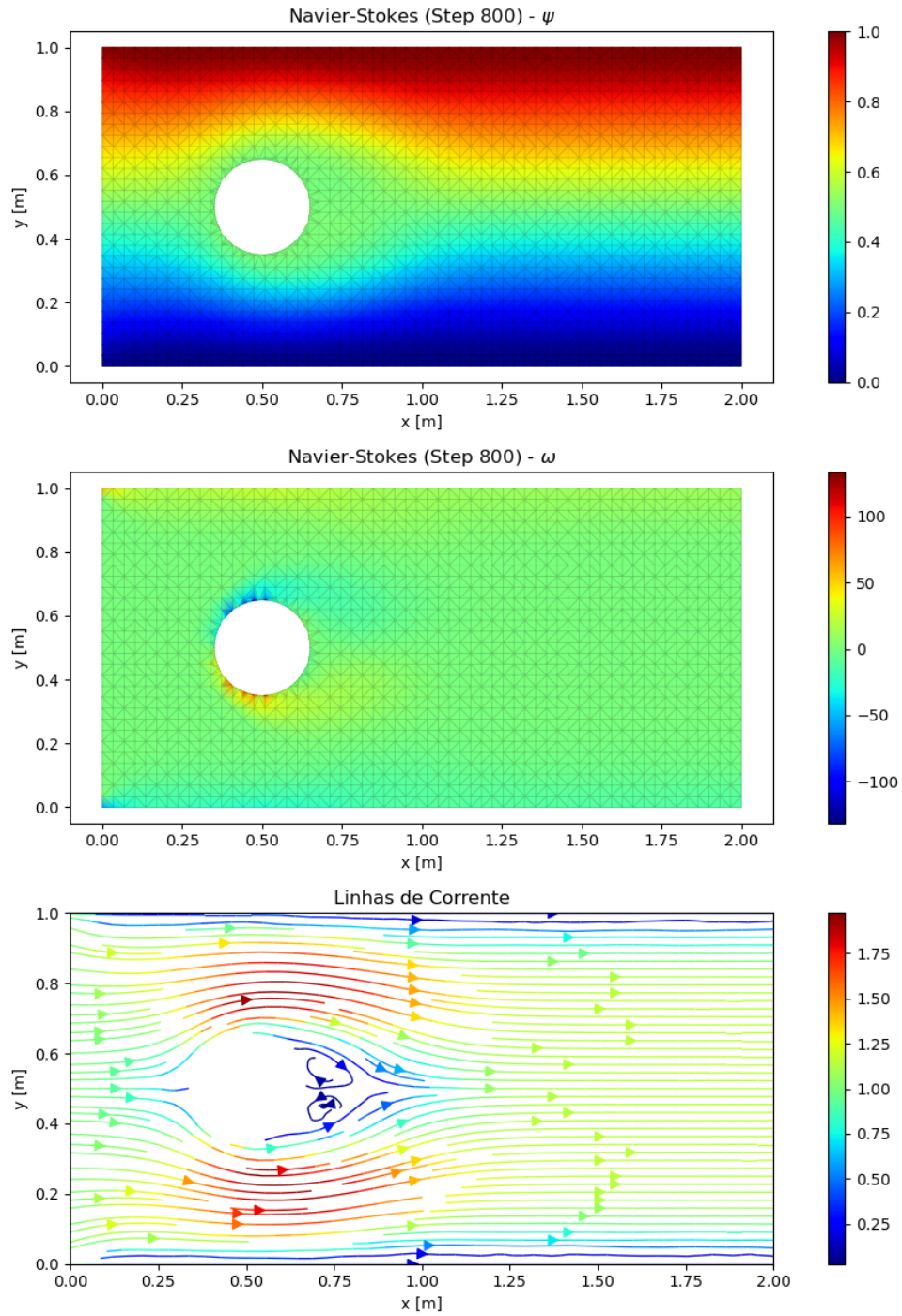


Figura 4.11: Navier-Stokes 2D — linhas de corrente para escoamento ao redor de um cilindro.

4.5 Síntese do Capítulo

Este capítulo apresentou a arquitetura e a implementação do *MinervaMesh*. A escolha do PyScript e da computação *client-side* viabiliza uma plataforma de

simulação numérica com custo de infraestrutura zero, portabilidade total e código aberto para inspeção acadêmica. A organização modular do repositório — com cada simulação autocontida em três arquivos (HTML, Python, TOML) — permite a extensão da plataforma com novos casos sem impacto nos módulos existentes. Os oito módulos MEF descritos cobrem três domínios da Engenharia Mecânica (térmico, estrutural e fluidodinâmico), demonstrando a versatilidade da formulação de elementos finitos apresentada no Capítulo 3. O Capítulo 5 apresentará a validação quantitativa desses módulos através da comparação com soluções analíticas e benchmarks da literatura.

Capítulo 5

Resultados e Validação

5.1 Introdução ao Capítulo

Este capítulo é dedicado à apresentação e discussão dos resultados obtidos com o *MinervaMesh*. A validação é realizada comparando os resultados numéricos com soluções analíticas e benchmarks da literatura, conforme estabelecido no plano de trabalho.

Capítulo 6

Conclusões

6.1 Conclusões

Este trabalho apresentou o desenvolvimento e a validação do *MinervaMesh*, uma plataforma *client-side* para simulação numérica em Engenharia Mecânica. Os resultados obtidos demonstram a viabilidade do uso de tecnologias web modernas para a execução de métodos numéricos com rigor acadêmico.

6.2 Trabalhos Futuros

Como extensões deste trabalho, propõe-se a implementação de elementos de ordem superior, a inclusão de modelos de turbulência e a otimização do solver para problemas tridimensionais de maior escala.

Referências Bibliográficas

- [1] MALISKA, C. R., *Transferência de Calor e Mecânica dos Fluidos Computacional*. 2 ed. Rio de Janeiro, LTC, 2004.
- [2] VERSTEEG, H. K., MALALASEKERA, W., *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. 2 ed. Harlow, Pearson Education, 2007.
- [3] ANJOS, G. R., *Computação Científica para Engenheiros*. Rio de Janeiro, PEM/COPPE/UFRJ, 2013. Notas de Aula.
- [4] PyScript Development Team, “PyScript Documentation”, Disponível em: <https://pyscript.net>, 2023, Acesso em: 20 fev. 2026.
- [5] HARRIS, C. R., MILLMAN, K. J., WALT, S. J. V. D., *et al.*, “Array programming with NumPy”, *Nature*, v. 585, n. 7825, pp. 357–362, 2020.
- [6] BROOKS, A. N., HUGHES, T. J. R., “Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations”, *Computer Methods in Applied Mechanics and Engineering*, v. 32, n. 1-3, pp. 199–259, 1982.
- [7] FISH, J., BELYTSCHKO, T., *A First Course in Finite Elements*. West Sussex, John Wiley & Sons, 2007.
- [8] REDDY, J. N., *An Introduction to the Finite Element Method*. 3 ed. New York, McGraw-Hill, 2005.
- [9] INCROPERA, F. P., DEWITT, D. P., BERGMAN, T. L., *et al.*, *Fundamentals of Heat and Mass Transfer*. 6th ed. Hoboken, John Wiley & Sons, 2008.

- [10] OZISIK, M. N., *Heat Conduction*. 2 ed. New York, John Wiley & Sons, 1993.
- [11] CENGEL, Y. A., GHAJAR, A. J., *Heat and Mass Transfer: Fundamentals and Applications*. 5 ed. New York, McGraw-Hill Education, 2015.
- [12] BEJAN, A., *Heat Transfer*. New York, Wiley, 1992.
- [13] FOX, R. W., MCDONALD, A. T., PRITCHARD, P. J., *Introdução à Mecânica dos Fluidos*. 7th ed. Rio de Janeiro, LTC, 2010.
- [14] WHITE, F. M., *Viscous Fluid Flow*. 3 ed. New York, McGraw-Hill, 2006.
- [15] BIRD, R. B., STEWART, W. E., LIGHTFOOT, E. N., *Transport Phenomena*. 2 ed. New York, John Wiley & Sons, 2002.
- [16] PATANKAR, S. V., *Numerical Heat Transfer and Fluid Flow*. New York, Hemisphere Publishing Corporation, 1980.
- [17] LEWIS, R. W., NITHIARASU, P., SEETHARAMU, K. N., *Fundamentals of the Finite Element Method for Heat and Fluid Flow*. Chichester, John Wiley & Sons, 2004.
- [18] ZIENKIEWICZ, O. C., TAYLOR, R. L., ZHU, J. Z., *The Finite Element Method: Its Basis and Fundamentals*. 7th ed. Oxford, Butterworth-Heinemann, 2013.
- [19] BATHE, K.-J., *Finite Element Procedures*. Upper Saddle River, Prentice Hall, 1996.
- [20] LOGAN, D. L., *A First Course in the Finite Element Method*. 4 ed. Belmont, Thomson, 2007.
- [21] HUEBNER, K. H., DEWHIRST, D. L., SMITH, D. E., *et al.*, *The Finite Element Method for Engineers*. 4 ed. New York, John Wiley & Sons, 2001.
- [22] GALERKIN, B. G., “Series in some questions of elastic equilibrium of beams and plates”, *Vestnik Inzhenerov i Tekhnikov*, v. 19, pp. 897–908, 1915.
- [23] HRENNIKOFF, A., “Solution of problems of elasticity by the framework method”, *Journal of Applied Mechanics*, v. 8, pp. 169–175, 1941.

- [24] COURANT, R., “Variational methods for the solution of problems of equilibrium and vibrations”, *Bulletin of the American Mathematical Society*, v. 49, n. 1, pp. 1–23, 1943.
- [25] TURNER, M. J., CLOUGH, R. W., MARTIN, H. C., *et al.*, “Stiffness and deflection analysis of complex structures”, *Journal of the Aeronautical Sciences*, v. 23, n. 9, pp. 805–823, 1956.
- [26] CLOUGH, R. W., “The finite element method in plane stress analysis”. In: *Proceedings of the 2nd ASCE Conference on Electronic Computation*, Pittsburgh, 1960.
- [27] DONEA, J., “A Taylor-Galerkin method for convective transport problems”, *International Journal for Numerical Methods in Engineering*, v. 20, n. 1, pp. 101–119, 1984.
- [28] CHRISTIE, I., GRIFFITHS, D. F., MITCHELL, A. R., *et al.*, “Finite element methods for second order differential equations with significant first derivatives”, *International Journal for Numerical Methods in Engineering*, v. 10, pp. 1389–1396, 1976.
- [29] LÖHNER, R., MORGAN, K., ZIENKIEWICZ, O. C., “The solution of non-linear hyperbolic equation systems by the finite element method”, *International Journal for Numerical Methods in Fluids*, v. 4, pp. 1043–1063, 1984.
- [30] GHIA, U., GHIA, K. N., SHIN, C. T., “High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method”, *Journal of Computational Physics*, v. 48, pp. 387–411, 1982.
- [31] MARCHI, C. H., SUERO, R., ARAKI, L. K., “The lid-driven square cavity flow: numerical solution with a 1024 x 1024 grid”, *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, v. 31, n. 3, pp. 186–198, 2009.
- [32] THOMAS, C., MORGAN, K., TAYLOR, C., “A finite element analysis of flow over a backward facing step”, *Computers & Fluids*, v. 9, n. 3, pp. 265–278, 1981.

- [33] ARMALY, B. F., DURST, F., PEREIRA, J. C. F., *et al.*, “Experimental and theoretical investigation of backward-facing step flow”, *Journal of Fluid Mechanics*, v. 127, pp. 473–496, 1983.
- [34] PIRONNEAU, O., “On the transport-diffusion algorithm and its applications to the Navier-Stokes equations”, *Numerische Mathematik*, v. 38, pp. 309–332, 1982.
- [35] GEUZAIN, C., REMACLE, J.-F., “Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities”, *International Journal for Numerical Methods in Engineering*, v. 79, n. 11, pp. 1309–1331, 2009.
- [36] AGUIAR, G. D. S. O. N. D., *Simulação Numérica da Transferência de Calor em um Disco de Freio Durante Frenagem usando o Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [37] ALVES, F. R. D. M., *Solução da equação tridimensional transiente do calor através do método de elementos finitos aplicado a discos de freio*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [38] FERREIRA, G. P., *Criação de um código em Python para simular a transferência de calor em um processador computacional*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [39] PINHEIRO, V. G., *Análise de condução de calor em componentes eletrônicos de material heterogêneo utilizando método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [40] CAPITANIO, T. A., *Implementação do método de elementos finitos em Python para o estudo paramétrico de dissipadores de calor de processadores computacionais*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [41] MIRANDA, L. M., *Estudo de geometrias de canais em placa fria para arrefecimento de eletrônicos através do método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.

- [42] TEIXEIRA, M. M., *Solução da equação de Laplace tridimensional para a temperatura aplicada a um bloco de motor através do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [43] OLIVEIRA, M. D. D., *Estudo da transferência de calor em dissipador de calor de aletas retas por meio do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [44] ARAUJO, R. S. D. A., *Comparação e validação de métodos numéricos aplicados à Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [45] FLORENTINO, Y. D. S., *Estudo de escoamentos para arrefecimento de eletrônicos através de Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [46] SANTOS, J. D. O. D., *Investigação dos campos de velocidade, vorticidade e função corrente em escoamento livre utilizando o Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [47] SILVA, J. A., *Método de Elementos Finitos em Python com a abordagem lagrangiana-euleriana para as oscilações de um cilindro*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [48] AMARAL, T. A. C. D., *Análise CFD de difusão de espécie química para diferentes geometrias de stent farmacológico*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [49] GOMES, M. F., *Análise CFD de escoamento em artérias coronárias para diferentes configurações de restrição de fluxo*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [50] GOMES, Y. P., *Análise Variacional e Numérica da Equação de Poisson em Problemas de Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.

Apêndice A

O que é um apêndice

Elemento que consiste em um texto ou documento elaborado pelo autor, com o intuito de complementar sua argumentação, sem prejuízo do trabalho. São identificados por letras maiúsculas consecutivas e pelos respectivos títulos.

Apêndice B

Encadernação do Projeto de Graduação

Número	UNIVERSIDADE FEDERAL DO RIO DE JANEIRO Escola Politécnica Departamento de Eletrônica e de Computação
Nome do Aluno	
Título do Projeto*	Título do Projeto Nome do Aluno
Ano	Projeto de Graduação Mês / Ano

*** Título resumido caso necessário**
Capa na cor preta, inscrições em dourado

Figura B.1: Encadernação do projeto de graduação.

Apêndice C

O que é um anexo

Documentação não elaborada pelo autor, ou elaborada pelo autor mas constituindo parte de outro projeto.